

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
Тема: Расстояние Левенштейна
Вариант 146

Студент гр. 4345

Кравцов С. А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритм поиска расстояния Левенштейна.

Постановка задачи

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример:

Для строк `pedestal` и `stien` расстояние Левенштейна равно 7:

- Сначала нужно совершить четыре операции удаления символа:
`pedestal` $\rightarrow$ `stal`.
- Затем необходимо заменить два последних символа: `stal` $\rightarrow$ `stie`.
- Потом нужно добавить символ в конец строки: `stie` $\rightarrow$ `stien`.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. ($S, 1 \leq |S| \leq 2550$).

Вторая строка входных данных содержит строку из строчных латинских букв. ($T, 1 \leq |T| \leq 2550$).

Параметры выходных данных:

Одно число L , равное расстоянию Левенштейна между строками S и T .

Sample Input:

`pedestal`

`stien`

Sample Output:

7

Вариант 146 - Методом динамического программирования вычислить: длину наибольшей общей подстроки двух строк, вывести и саму эту подстроку.

Выполнение работы

Для реализации алгоритма вычисления расстояния Левенштейна нужна матрица размера $n \times m$, и столько же операций сравнения. Но для заполнения каждой текущей строки матрицы нужна только предыдущая $\Rightarrow$ можно ограничиться двумя строками $\Rightarrow$ затраты памяти снизятся с $O(M \cdot N)$ до $O(\min(M, N))$, количество операций сравнения не изменится.

В общем случае на первом шаге строка `prev_row` заполняется ценой операции удаления или вставки для каждого символа т.е. числами 0, `del_cost`, `2del_cost`, ..., `mdel_cost` или `ins_cost`. Также в общем случае на каждой итерации 0-ой элемент `cur_row` = `i*ins_cost` или `del_cost`, где `i` – номер текущей итерации.

В нашем случае цена удаления, вставки, или замены равна 1.

В дальнейшем в заполнении `cur_row` используется следующая логическая цепочка:

- Если сравниваемые символы строк совпадают, то `cur_row[j]` = `prev_row[j-1]`, т.е. верхний левый элемент матрицы – не нужно производить операции для получения строки.
- Если сравниваемые символы строк не совпадают, то `cur_row[j]` = $\min(\text{prev_row}[j], \text{prev_row}[j-1], \text{cur_row}[j-1]) + 1$, т.е. минимум из верхнего, левого и левого верхнего элементов + 1 – минимальной стоимостью будет лучший из предыдущих вариантов + одна операция для изменения текущего элемента.

В соответствии с вариантом 14б основной алгоритм был дополнен. Логика дополнения заключается в ведении вспомогательной строки `cur_sub`, которая увеличивает значение на 1 только при совпадении символов и обнуляется при любом несовпадении. Это позволяет отслеживать длину самой длинной непрерывной цепочки совпадений, фиксируя её конечный индекс для последующего извлечения подстроки через срез.

Сложность алгоритма по времени $O(M \cdot N)$, обусловлена сравнения каждого символа первой строки с каждым символом второй строки.

Сложность алгоритма по памяти $O(\min(M, N))$, обусловлена хранением двух строк, а не матрицы размера m на n .

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1.

Таблица 1 – Результаты тестирования

№	Входные данные	Выходные данные	Комментарий
1	pedestal stien	7 st 2	Пример из задания
2	entrance reenterable	5 ent 3	Пример из задания
3	asdzxc asdzxc	0 asdzxc 6	Совпадающие строки
4	asdzxc sdzxca	2 sdzxc 5	Сдвиг
5	арбузибарабан пузобарабана	5 барабан 7	Несколько подстрок

Выводы

В ходе работы были реализованы алгоритмы динамического программирования для вычисления расстояния Левенштейна и поиска наибольшей общей подстроки. Использование двух пар вспомогательных массивов позволило оптимизировать потребление памяти до линейной зависимости $O(\min(M, N))$. Оба показателя вычисляются синхронно в рамках одного двойного цикла, что минимизирует временные затраты при обработке длинных строк.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.py

```
a = input()
b = input()

if len(a) > len(b):
    a, b = b, a

n = len(a)
m = len(b)

prev_row = [i for i in range(n + 1)]
cur_row = [0] * (n + 1)

prev_sub = [0] * (n + 1)
cur_sub = [0] * (n + 1)

max_len = 0
end_pos = 0

for i in range(1, m + 1):
    cur_row[0] = i

    print(f"Шаг {i}: символ b: '{b[i-1]]':")

    for j in range(1, n + 1):
        print(f"  Сравнение '{b[i-1]]' и '{a[j-1]]': ", end="")
        if a[j-1] == b[i-1]:
            cur_row[j] = prev_row[j-1]
            print(f"Совпадение   cur_row[{j}]   =   prev_row[{j-1}]   =
{prev_row[j-1]}")
        else:
            cost = min(prev_row[j], prev_row[j-1], cur_row[j-1]) + 1
            cur_row[j] = cost
            print(f"Несовпадение cur_row[{j}] = {cost}")
```

```

    if a[j-1] == b[i-1]:
        cur_sub[j] = prev_sub[j-1] + 1

        if cur_sub[j] > max_len:
            max_len = cur_sub[j]
            end_pos = j
    else:
        cur_sub[j] = 0

    print(f"    prev_row - {prev_row}")
    print(f"    cur_row: {cur_row}")

    prev_row[:] = cur_row[:]
    prev_sub[:] = cur_sub[:]

lev_distance = prev_row[n]
substring = a[end_pos - max_len:end_pos]

print(lev_distance)
print(substring, max_len)

```